

EXPERIMENT 14

Alpha Beta Pruning

Aim

To implement the **Alpha-Beta Pruning Algorithm** for a two-player gaming problem using Python.

Algorithm

1. Start with the initial game state.
2. Generate all possible moves.
3. Set **Alpha** = $-\infty$ and **Beta** = $+\infty$.
4. For the **Max player**, select the maximum value.
5. For the **Min player**, select the minimum value.
6. Update Alpha for Max and Beta for Min.
7. If **Alpha** $\geq$ **Beta**, prune the remaining branches.
8. Continue until the best move is obtained.
9. Display the optimal score.

Program

```
def alpha_beta(depth, node, maximizing, values, alpha, beta, height):
```

```
    # Terminal node
```

```
    if depth == height:
```

```
        return values[node]
```

```
    if maximizing:
```

```
        best = -float("inf")
```

```

for i in range(2):

    value = alpha_beta(

        depth + 1,

        node * 2 + i,

        False,

        values,

        alpha,

        beta,

        height

    )

    best = max(best, value)

    alpha = max(alpha, best)

    # Beta cutoff

    if beta <= alpha:

        break

return best

else:

    best = float("inf")

```

```
for i in range(2):

    value = alpha_beta(

        depth + 1,

        node * 2 + i,

        True,

        values,

        alpha,

        beta,

        height

    )

    best = min(best, value)

    beta = min(beta, best)

# Alpha cutoff

if beta <= alpha:

    break

return best
```

```
# Input
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
height = 3
```

```
# Alpha-Beta Pruning
```

```
best_score = alpha_beta(
```

```
    0, 0, True, values,
```

```
    -float("inf"),
```

```
    float("inf"),
```

```
    height
```

```
)
```

```
print("Game values:", values)
```

```
print("Best score using Alpha-Beta Pruning:", best_score)
```

Input

Game values:

3 5 2 9 12 5 23 23

Output

```
Game values: [3, 5, 2, 9, 12, 5, 23, 23]
Best score using Alpha-Beta Pruning: 12
```

Result

Thus, the **Alpha-Beta Pruning Algorithm** was successfully implemented in Python, and the optimal game score was obtained as **12** while avoiding unnecessary branches.

